Desenvolvimento de um Serviço de Metadados Compatível com Cloud-Init para Incus

Gustavo Santos[1], **Maycon L. M. Peixoto**[1]

[1]Departamento de Ciência da Computação – Universidade Federal da Bahia (UFBA)

{gustavoafs,mayconleone}@ufba.br

Abstract. *Public cloud providers offer instance metadata services that allow virtual machines and containers to discover their own configuration at boot time, enabling automated provisioning through cloud-init. However, private infrastructure managers such as Incus lack an equivalent native metadata service, forcing administrators to rely on manual configuration or ad-hoc solutions. This work presents the design and implementation of a cloud-init compatible metadata service for Incus, providing REST API endpoints for metadata, user-data, vendor-data, and network configuration delivery. The service uses an event-driven architecture with periodic synchronization to maintain up-to-date instance information in a local database, identifying requesting instances by their source IP address, and optionally supports high-availability deployments through Raft-based consensus. The proposed solution is evaluated in a real Incus environment using system containers on a single host, demonstrating functional compatibility with cloud-init, response latencies in the tens of milliseconds with up to 200 concurrent containers, and leader re-election in about two seconds across ten failover runs in a three-node cluster.*

Resumo. *Provedores de nuvem pública oferecem serviços de metadados de instância que permitem que máquinas virtuais e contêineres descubram sua própria configuração durante a inicialização, viabilizando o provisionamento automatizado por meio do cloud-init. No entanto, gerenciadores de infraestrutura privada como o Incus não possuem um serviço de metadata nativo equivalente, obrigando administradores a recorrer a configurações manuais ou soluções ad-hoc. Este trabalho apresenta o projeto e a implementação de um serviço de metadados compatível com cloud-init para Incus, fornecendo endpoints REST para entrega de metadados, user-data, vendor-data e configuração de rede. O serviço utiliza uma arquitetura orientada a eventos com sincronização periódica para manter informações atualizadas das instâncias em um banco de dados local, identificando as instâncias requisitantes pelo endereço IP de origem, e oferece, opcionalmente, alta disponibilidade por meio de consenso distribuído baseado em RAFT. A solução proposta é avaliada em um ambiente real com Incus, usando contêineres de sistema em um único host, demonstrando compatibilidade funcional com o cloud-init, latências de resposta na faixa de dezenas de milissegundos com até 200 contêineres simultâneos e reeleição de líder em cerca de dois segundos, em dez repetições de falha, em um cluster de três nós.*

1. Introdução

A computação em nuvem [Mell and Grance, 2011] transformou profundamente a forma como infraestruturas são provisionadas e gerenciadas. Provedores públicos como Amazon

Web Services (AWS), Google Cloud Platform (GCP) e Microsoft Azure consolidaram um modelo operacional no qual máquinas virtuais e contêineres são criados, configurados e descartados de forma programática e automatizada. Central a esse modelo está o *Instance Metadata Service* (IMDS), um serviço interno acessível por um endereço IP reservado, tipicamente `169.254.169.254`, pertencente à faixa de endereços *link-local* IPv4 definida pela RFC 3927 [Cheshire et al., 2005]. Por meio dele, cada instância descobre sua própria configuração em tempo de inicialização: nome do host, endereços IP, chaves SSH autorizadas e *scripts* de *user-data* [Amazon Web Services, 2024a]. Essa capacidade é o alicerce sobre o qual ferramentas de provisionamento automatizado, especialmente o *cloud-init*, operam para configurar instâncias sem intervenção manual [Canonical Ltd., 2024a].

Gerenciadores de contêineres e máquinas virtuais voltados à infraestrutura privada, como o Incus, projeto derivado (*fork*) do LXD mantido pela organização Linux Containers [Linux Containers, 2024, 2023], carecem de um serviço de metadados nativo equivalente ao oferecido pelas nuvens públicas. Essa ausência obriga administradores a configurar instâncias manualmente ou a recorrer a *scripts* ad hoc que não seguem nenhum padrão estabelecido, rompendo o fluxo de provisionamento esperado pelo *cloud-init* e inviabilizando a portabilidade de configurações entre ambientes públicos e privados. O resultado prático é que ambientes Incus, por mais capazes que sejam em termos de virtualização, tornam-se cidadãos de segunda classe no ecossistema de automação de infraestrutura.

Embora soluções como o serviço de metadados do OpenStack abordem esse problema no contexto de plataformas de nuvem privada completas [OpenStack Foundation, 2024], não foi identificada, na revisão de trabalhos relacionados realizada neste trabalho (Seção 2), uma solução leve e independente projetada especificamente para ambientes Incus. Isso é particularmente relevante para cenários de *homelab*, computação de borda (*edge computing*) [Shi et al., 2016] e nuvens privadas de pequena escala, nos quais implantar uma plataforma completa como o OpenStack representa uma complexidade operacional desproporcional às necessidades do ambiente. Há, portanto, uma lacuna técnica não atendida pelas soluções examinadas.

Este trabalho propõe o desenvolvimento de um serviço de metadados compatível com *cloud-init* para o Incus. O serviço implementa uma API REST que entrega metadados, *user-data*, *vendor-data* e configuração de rede às instâncias, seguindo a interface esperada pelo *cloud-init* [Canonical Ltd., 2024a]. A arquitetura adota um modelo orientado a eventos combinado com sincronização periódica, garantindo que os dados das instâncias permaneçam atualizados de forma consistente com o estado reportado pelo gerenciador Incus. O serviço foi concebido para ser implantado de forma autônoma, sem dependência de componentes externos de grande porte, atendendo diretamente ao perfil de ambientes de infraestrutura privada de menor escala. Para cenários que exigem tolerância a falhas, o serviço oferece, opcionalmente, alta disponibilidade por meio do algoritmo de consenso RAFT [Ongaro and Ousterhout, 2014], com múltiplos nós coordenados por eleição de líder e replicação de *log*.

As principais contribuições deste trabalho são: (a) uma análise das implementações de serviços de metadados em nuvens públicas e privadas, identificando os requisitos mínimos para compatibilidade com o *cloud-init*; (b) o projeto e a implementação de um serviço de metadados compatível com *cloud-init* para o Incus, incluindo a modelagem da

API, o mecanismo de sincronização de instâncias, a persistência dos dados e o suporte opcional a alta disponibilidade por consenso RAFT; e (c) a avaliação experimental do serviço em um ambiente Incus real com contêineres de sistema, cobrindo a corretude do provisionamento automatizado, a latência e a escalabilidade da API com até 200 contêineres simultâneos em um único host e o comportamento do mecanismo de alta disponibilidade em um *cluster* de três nós. O restante deste trabalho está organizado da seguinte forma: a Seção 2 apresenta os trabalhos relacionados; a Seção 3 detalha a proposta e a implementação; a Seção 4 apresenta a avaliação experimental; e a Seção 5 conclui o trabalho.

2. Trabalhos Relacionados

Esta seção apresenta os principais trabalhos e tecnologias relacionados ao serviço de metadados proposto neste trabalho. São discutidos o mecanismo de inicialização de instâncias cloud-init, os serviços de metadados das principais nuvens públicas e privadas, e as limitações atuais do Incus em relação a esse ecossistema.

2.1. Cloud-init e Inicialização de Instâncias

O cloud-init é o padrão da indústria para a inicialização automatizada de instâncias em ambientes de nuvem [Canonical Ltd., 2024a]. Desenvolvido originalmente pela Canonical e amplamente adotado pelos principais provedores de infraestrutura, o cloud-init é executado durante a inicialização do sistema operacional e é responsável por configurar aspectos fundamentais da instância, tais como nome de host, usuários e grupos, chaves SSH, pacotes de software, scripts de inicialização e parâmetros de rede.

O processo de inicialização do cloud-init é organizado em quatro estágios sequenciais, cada um com responsabilidades bem definidas [Canonical Ltd., 2024a]: (1) `init-local`, executado antes da configuração de rede, responsável por identificar o *datasource* a partir de fontes locais como disco, DMI e linha de comando do *kernel*; (2) `init` (rede), executado após a rede estar ativa, responsável por buscar os dados de configuração do *datasource* identificado; (3) `modules-config`, que aplica os módulos de configuração como criação de usuários e instalação de pacotes; e (4) `modules-final`, que executa *scripts* finais e mensagens de conclusão. Essa sequência é relevante porque impõe requisitos temporais à disponibilidade do serviço de metadados: os dados devem estar acessíveis antes do estágio `init`, momento em que o cloud-init realiza a busca efetiva dos metadados.

O funcionamento do cloud-init é baseado no conceito de *datasources*: fontes de dados a partir das quais a ferramenta obtém as informações necessárias para configurar a instância. Cada datasource é composto por quatro tipos de dados distintos: (i) *metadata*, que contém informações de identidade da instância como identificador único, nome de host e dados de rede; (ii) *user-data*, que permite ao usuário final fornecer configurações personalizadas, como *scripts* shell ou arquivos de configuração no formato cloud-config; (iii) *vendor-data*, fornecido pelo provedor de infraestrutura para configurações específicas da plataforma; e (iv) *network-config*, que define a configuração de rede da instância em formatos padronizados [Canonical Ltd., 2024a]. Dentre esses dados, o campo `instance-id` é particularmente crítico: o cloud-init utiliza-o para determinar se a inicialização corrente é a primeira (*first boot*), acionando a configuração completa, ou uma inicialização subsequente, na qual módulos já aplicados são ignorados.

A capacidade do cloud-init de abstrair as particularidades de cada plataforma por meio da camada de datasources é o que o torna uma solução portável e amplamente adotada. No entanto, para que essa abstração funcione corretamente, é necessário que a infraestrutura subjacente disponibilize um endpoint de metadados que o cloud-init possa consultar durante a inicialização.

2.2. Datasource NoCloud

O datasource *NoCloud* é o mais flexível dentre os oferecidos pelo cloud-init e é diretamente relevante para ambientes de infraestrutura privada [Canonical Ltd., 2024f]. Ele permite obter dados de configuração a partir de uma URL HTTP base (`seedfrom`), consultando os caminhos `/meta-data`, `/user-data` e `/vendor-data` relativos a essa URL; o quarto tipo de dado, `network-config`, é lido pelo *NoCloud* apenas de fontes locais (disco ou sistema de arquivos), e não pela URL `seedfrom`, conforme verificado na Seção 4.2 para a versão do cloud-init utilizada neste trabalho. Essa abordagem não exige que o serviço implemente a hierarquia de caminhos versionados do EC2 (como `/latest/meta-data/`), bastando que os endpoints estejam disponíveis na raiz configurada. O *NoCloud* suporta os protocolos HTTP, HTTPS, FTP e FTPS, e requer apenas que o arquivo `meta-data` contenha, no mínimo, os campos `instance-id` e `local-hostname` [Canonical Ltd., 2024f]. Essa simplicidade torna o *NoCloud* especialmente adequado para serviços de metadados independentes de plataforma, como o proposto neste trabalho.

2.3. AWS Instance Metadata Service (IMDS)

A Amazon Web Services (AWS) disponibiliza o *Instance Metadata Service* (IMDS), acessível pelas instâncias EC2 por meio do endereço IP de link-local `169.254.169.254` [Amazon Web Services, 2024a]. Esse serviço expõe uma API HTTP que permite às instâncias consultar informações sobre si mesmas sem necessidade de credenciais adicionais.

O IMDS oferece duas versões com características distintas de segurança. A primeira versão, IMDSv1, opera com requisições GET simples sem autenticação, o que pode expor a instância a ataques de falsificação de requisição do lado do servidor (*Server-Side Request Forgery*, SSRF) [MITRE Corporation, 2021]. A segunda versão, IMDSv2, introduzida em 2019, adota um modelo baseado em sessões: a instância deve primeiro obter um token de acesso por meio de uma requisição PUT com cabeçalho `X-aws-ec2-metadata-token-ttl-seconds`, e em seguida utilizar esse token nas requisições subsequentes via o cabeçalho `X-aws-ec2-metadata-token` [Amazon Web Services, 2024b]. Essa abordagem mitiga significativamente os riscos associados a ataques SSRF, uma vez que a maioria das vulnerabilidades SSRF permite apenas requisições GET e não possibilita a inclusão de cabeçalhos personalizados. Além disso, o IMDSv2 aplica um limite de saltos (*hop limit*) de 1 nas respostas PUT e rejeita requisições contendo o cabeçalho `X-Forwarded-For`, bloqueando ataques baseados em proxies reversos [Amazon Web Services, 2019].

Os dados servidos pelo IMDS incluem, entre outros: identificador único da instância (`instance-id`), nome de host, endereços MAC das interfaces de rede, metadados de interfaces de rede, chaves SSH públicas associadas à instância e o *user-data* fornecido

no momento do lançamento. A API é organizada em um namespace hierárquico acessível via caminhos como `/latest/meta-data/` e `/latest/user-data`.

2.4. GCP Metadata Server

O Google Cloud Platform (GCP) disponibiliza o *Metadata Server*, acessível pelo nome de host `metadata.google.internal` (resolvido para `169.254.169.254`) e também diretamente pelo endereço IP `169.254.169.254` [Google Cloud, 2024]. O serviço é organizado em dois escopos principais: metadados de projeto, que contêm informações compartilhadas entre todas as instâncias de um projeto, e metadados de instância, específicos de cada máquina virtual.

Um aspecto de segurança relevante do Metadata Server do GCP é a exigência do cabeçalho HTTP `Metadata-Flavor: Google` em todas as requisições. Requisições que não incluam esse cabeçalho são rejeitadas pelo serviço, o que serve como mecanismo de proteção contra ataques SSRF oriundos de contextos externos à instância [Google Cloud, 2024]. Mecanismo semelhante é adotado pelo Azure Instance Metadata Service, que exige o cabeçalho `Metadata: true` [Microsoft, 2024]. Embora esses cabeçalhos obrigatórios mitiguem ataques SSRF simples, que tipicamente não permitem a inclusão de cabeçalhos personalizados, eles oferecem proteção inferior ao modelo de sessões do IMDSv2 da AWS [Amazon Web Services, 2019]. Além da proteção contra SSRF, o serviço do GCP suporta notificações de alterações de metadados via mecanismo de *long polling*, permitindo que aplicações reajam dinamicamente a mudanças de configuração.

2.5. OpenStack Metadata Service

O OpenStack, principal plataforma de nuvem privada de código aberto, implementa um serviço de metadados compatível com a API EC2 da AWS, também disponível no endereço `169.254.169.254` [OpenStack Foundation, 2024]. O serviço oferece dois endpoints distintos: o endpoint compatível com EC2 (`/latest/meta-data/`) e o endpoint nativo do OpenStack (`/openstack/`), que serve dados em formato JSON estruturado conforme o modelo de dados da plataforma.

Uma característica fundamental do serviço de metadados do OpenStack é seu forte acoplamento com os demais componentes da plataforma. O roteamento de requisições de metadados é realizado pelo Neutron, o componente de rede do OpenStack, que intercepta requisições destinadas ao endereço `169.254.169.254` e as encaminha ao serviço correto com base na interface de rede da instância solicitante. O provisionamento dos dados de metadados, por sua vez, é gerenciado pelo Nova, o componente de computação. Esse acoplamento implica que o serviço de metadados do OpenStack não pode ser executado de forma autônoma fora do ecossistema OpenStack [OpenStack Foundation, 2024], o que limita sua aplicabilidade em outros contextos de virtualização. Em contextos acadêmicos, a dependência do ecossistema completo é aceita quando a escala justifica: Bollig et al. [2018] descrevem a plataforma Stratus, uma nuvem privada baseada em OpenStack na Universidade de Minnesota, na qual o cloud-init consulta o serviço de metadados do Nova durante a inicialização para configurar endereços IP, chaves SSH e, por meio do *vendor-data*, aplicar atualizações de segurança obrigatórias em todas as máquinas virtuais. Esse exemplo ilustra como o padrão de metadata service é fundamental para operações de nuvem privada, mas também evidencia que sua adoção requer a implantação de uma plataforma completa como o OpenStack.

2.6. Incus/LXD e a Lacuna de Metadados

O Incus é um gerenciador moderno de contêineres de sistema e máquinas virtuais, surgido em 2023 como um *fork* da comunidade do projeto LXD, originalmente desenvolvido e mantido pela Canonical [Linux Containers, 2023, 2024]. O *fork* foi criado por Aleksa Sarai e incorporado ao projeto Linux Containers em agosto de 2023, pouco depois de a Canonical retirar o LXD desse projeto [Linux Containers, 2023]. Stéphane Graber, criador original do LXD, passou então a liderar o desenvolvimento do Incus, cuja primeira versão estável foi publicada em outubro de 2023 [Graber, 2023]. O Incus herda a arquitetura e as funcionalidades do LXD [Canonical Ltd., 2024d], com o objetivo de manter o desenvolvimento aberto e orientado pela comunidade [Linux Containers, 2023].

O Incus possui suporte nativo para configurações do cloud-init por meio de chaves de configuração específicas: `cloud-init.user-data` para dados de usuário, `cloud-init.vendor-data` para dados do fornecedor e `cloud-init.network-config` para configuração de rede [Linux Containers, 2024]. Essas configurações podem ser definidas diretamente na instância ou propagadas via perfis de configuração. No entanto, o Incus não disponibiliza um endpoint de metadados acessível via rede (TCP/IP) que as instâncias possam consultar durante a inicialização.

A entrega das configurações do cloud-init no Incus ocorre por meio do *datasource LXD*, que se comunica com o Incus no host por uma API HTTP exposta em um *socket* Unix, em `/dev/incus/sock` (ou `/dev/lxd/sock` no LXD original) [Canonical Ltd., 2024c]. Em contêineres, esse *socket* é montado diretamente pelo host; em máquinas virtuais, ele é provido pelo *Incus agent*, um processo leve executado dentro da instância. Alternativamente, as configurações podem ser injetadas diretamente no sistema de arquivos da instância via *config drive* [Linux Containers, 2024]. Embora esses mecanismos sejam funcionais para casos de uso básicos, eles apresentam uma limitação temporal crítica: o *Incus agent* é iniciado tardiamente na sequência de *boot* da máquina virtual, de modo que os estágios iniciais do cloud-init (`ds-identify` e `init-local`) frequentemente são executados antes que o *socket* esteja disponível. Essa condição de corrida, relatada pela própria comunidade do projeto [Linux Containers Community, 2024], resulta em falhas na detecção do datasource e, consequentemente, na configuração automática da instância.

Trabalhos como o de Sousa [2018], que implementa uma infraestrutura de nuvem privada na USP utilizando LXD com OpenNebula para instanciação de contêineres sob demanda, evidenciam que a plataforma é viável para uso acadêmico e de pesquisa, mas a inicialização das instâncias depende de *scripts* definidos manualmente nos *templates* da plataforma de orquestração, sem um serviço de metadados padronizado.

Além dessa limitação temporal, os mecanismos atuais do Incus não replicam o fluxo completo do datasource do cloud-init que opera via requisições HTTP ao endereço `169.254.169.254`. Isso impede o uso do datasource *NoCloud* HTTP [Canonical Ltd., 2024f] ou do datasource *EC2* [Canonical Ltd., 2024b], que são os mais amplamente suportados e testados no ecossistema cloud-init, e limita a compatibilidade com imagens e ferramentas de automação que esperam um serviço de metadados acessível via rede.

2.7. Algoritmo de Consenso RAFT

O RAFT é um algoritmo de consenso distribuído projetado com ênfase na compreensibilidade, proposto por Ongaro e Ousterhout [2014]. Ao contrário do Paxos, que é re-

conhecidamente difícil de compreender e implementar corretamente, o RAFT decompõe o problema do consenso em subproblemas relativamente independentes: eleição de líder, replicação de log e segurança. Essa decomposição facilita tanto o entendimento do algoritmo quanto a construção de implementações corretas.

No RAFT, um cluster é composto por um nó líder e um ou mais seguidores. O líder é responsável por receber requisições de escrita, replicá-las para os seguidores via entradas de log, e confirmar a operação assim que uma maioria de nós tiver persistido a entrada. Essa garantia de quórum assegura que o sistema tolera falhas de até $\lfloor (n-1)/2 \rfloor$ nós em um cluster de n membros, mantendo disponibilidade e consistência.

A biblioteca `hashicorp/raft` [HashiCorp, 2024] é uma implementação em Go amplamente utilizada do protocolo RAFT, desenvolvida e mantida pela HashiCorp. Ela fornece uma API estável para integração do consenso distribuído em aplicações, sendo empregada em projetos de destaque como o Consul e o Vault. Neste trabalho, a biblioteca é utilizada para prover alta disponibilidade ao serviço de metadados, garantindo que o estado do cluster permaneça consistente mesmo diante de falhas de nós individuais.

2.8. Segurança em Serviços de Metadados

Serviços de metadados representam uma superfície de ataque relevante em ambientes de nuvem, uma vez que expõem informações sensíveis, como credenciais temporárias, chaves SSH e configurações de rede, por meio de um endpoint HTTP acessível sem autenticação convencional. O vetor de ataque mais significativo é o *Server-Side Request Forgery* (SSRF): caso uma aplicação web executada dentro da instância possua uma vulnerabilidade que permita a um atacante induzir requisições HTTP arbitrárias, o atacante pode direcionar essas requisições ao endereço `169.254.169.254` e obter os metadados da instância [MITRE Corporation, 2021]. Esse vetor é catalogado pela MITRE ATT&CK como a técnica T1552.005 (*Unsecured Credentials: Cloud Instance Metadata API*).

Um caso emblemático que ilustra a gravidade dessa classe de ataque é a violação de dados da Capital One, ocorrida em março de 2019 e analisada em profundidade por Novaes Neto et al. [2020]. Nesse incidente, um atacante explorou uma vulnerabilidade SSRF em um *Web Application Firewall* (WAF) mal configurado para induzir requisições ao IMDSv1 da AWS no endereço `169.254.169.254`, obtendo credenciais temporárias IAM associadas à instância. Com essas credenciais, o atacante acessou buckets S3 contendo dados de 106 milhões de clientes, resultando em prejuízos superiores a 150 milhões de dólares. A cadeia de ataque (SSRF, acesso ao IMDS sem autenticação, exfiltração de credenciais) demonstrou que a simplicidade do IMDSv1, baseado em requisições GET sem qualquer verificação, representava um risco sistêmico. Esse incidente foi um dos catalisadores para a introdução do IMDSv2 pela AWS.

Os provedores de nuvem adotam diferentes estratégias de mitigação. A AWS introduziu o IMDSv2 com autenticação baseada em sessões [Amazon Web Services, 2019], o GCP exige o cabeçalho `Metadata-Flavor: Google` [Google Cloud, 2024], e o Azure requer o cabeçalho `Metadata: true` [Microsoft, 2024]. Em ambientes de infraestrutura privada, como os atendidos pelo serviço proposto neste trabalho, o serviço de metadados não está exposto à internet e o perímetro de rede é controlado pelo administrador. Isso reduz a exposição externa, mas não elimina o risco de SSRF, cujo vetor é a própria instância: uma aplicação vulnerável executada dentro do *guest* pode ser induzida a requisitar o

endpoint de metadados pela rede interna, exatamente como no caso da Capital One, sem que o serviço precise estar acessível de fora. O serviço proposto, na versão avaliada, não exige cabeçalho nem token nas requisições, situando-se, nesse aspecto, no mesmo nível do IMDSv1, e não dos mecanismos do IMDSv2, do `Metadata-Flavor: Google` e do `Metadata: true`. A identificação por endereço IP de origem limita os dados que cada instância pode obter aos seus próprios, mas não impede que uma aplicação vulnerável dentro da instância os obtenha. A autenticação por token, registrada como trabalho futuro na Seção 5, é a mitigação correspondente.

2.9. Comparação entre as Abordagens

A Tabela 1 sintetiza as principais características dos serviços de metadados discutidos nesta seção em comparação com a proposta deste trabalho. Os critérios avaliados são: operação autônoma (sem dependência de plataforma), compatibilidade com o cloud-init via endpoint HTTP, suporte a contêineres de sistema, suporte a máquinas virtuais, alta disponibilidade e disponibilidade como software de código aberto.

Tabela 1. Comparação entre serviços de metadados para cloud-init. ○: suportado pelo projeto, mas não avaliado experimentalmente.

Solução	Autônomo	Endpoint HTTP cloud-init	Suporta Contêineres	Suporta VMs	Alta Disp.	Código Aberto
AWS IMDS	✗	✓	✗	✓	✓	✗
GCP Metadata	✗	✓	✗	✓	✓	✗
OpenStack Metadata	✗	✓	✗	✓	✓	✓
LXD/Incus (atual)	✓	✗	✓	✓	✗	✓
Este trabalho	✓	✓	✓	○	✓	✓

2.10. Considerações Finais

A análise dos trabalhos relacionados evidencia uma lacuna tecnológica relevante: entre os trabalhos e tecnologias examinados, não foi identificado um serviço de metadados leve e autônomo para o Incus que implemente o protocolo HTTP de metadados esperado pelo cloud-init, habilitando o fluxo completo de datasource para contêineres e máquinas virtuais gerenciados por essa plataforma. Os serviços existentes nas nuvens públicas (AWS IMDS e GCP Metadata Server) são proprietários e estão acoplados às respectivas infraestruturas. O serviço de metadados do OpenStack, apesar de ser código aberto e compatível com cloud-init, depende integralmente do ecossistema OpenStack para funcionar, o que dificulta seu uso isolado. O próprio Incus, por sua vez, oferece mecanismos de entrega de configuração baseados no datasource LXD [Canonical Ltd., 2024c] que, além de não cobrirem o fluxo completo de consulta HTTP, sofrem de uma condição de corrida temporal durante a inicialização. A necessidade de ferramentas para inicialização automatizada de instâncias em nuvens de infraestrutura é documentada na literatura desde 2011,

quando Bresnahan et al. [2011] propuseram o cloudinit.d, uma ferramenta para lançamento coordenado de conjuntos de máquinas virtuais interdependentes em nuvens IaaS, evidenciando já naquele período a dificuldade de configurar instâncias de forma repetível e automatizada sem o suporte de serviços de metadados. A adoção crescente de práticas de Infraestrutura como Código (*Infrastructure as Code*, IaC) [Guerriero et al., 2019; Artač et al., 2017] reforça a importância de serviços de metadados padronizados para o provisionamento automatizado de instâncias. Nesse contexto, Teppan et al. [2022] observam que, dentre as categorias de ferramentas *cloud-native* catalogadas pela CNCF, a categoria de provisionamento é a que menos dispõe de projetos maduros para ambientes *on-premise*, o que corrobora a lacuna identificada neste trabalho para o ecossistema Incus.

Este trabalho propõe um serviço de metadados compatível com cloud-init, projetado especificamente para ambientes Incus, que opera de forma autônoma, foi projetado para contêineres e máquinas virtuais (a avaliação experimental, contudo, restringiu-se a contêineres de sistema; Seção 4), oferece alta disponibilidade via o algoritmo de consenso RAFT [Ongaro and Ousterhout, 2014] implementado pela biblioteca `hashicorp/raft` [HashiCorp, 2024], e é disponibilizado como software de código aberto. Dessa forma, preenche a lacuna identificada na revisão realizada e viabiliza o uso do ecossistema completo do cloud-init em infraestruturas baseadas em Incus.

3. Proposta

Este trabalho propõe o desenvolvimento de um serviço de metadados compatível com cloud-init voltado a instâncias gerenciadas pelo Incus [Linux Containers, 2024]. O objetivo é viabilizar a configuração automática de contêineres e máquinas virtuais no momento de sua inicialização, por meio do protocolo de metadados já consolidado pelo cloud-init [Canonical Ltd., 2024a], sem a necessidade de configuração manual por instância.

O cloud-init é amplamente utilizado em ambientes de nuvem pública para realizar tarefas como definição de hostname, injeção de chaves SSH, configuração de rede e execução de scripts na primeira inicialização. Contudo, o Incus não oferece nativamente um serviço de metadados acessível via HTTP pelas instâncias, o que impede o uso do cloud-init em seu modo de operação padrão baseado em datasource de rede. A proposta preenche essa lacuna ao expor uma API REST que as instâncias podem consultar durante o boot, identificando-se pelo próprio endereço IP de origem.

3.1. Arquitetura

O serviço segue uma arquitetura modular, com responsabilidades bem delimitadas entre seus componentes. A Figura 1 ilustra a interação entre os principais módulos do sistema.

Os componentes são organizados da seguinte forma:

- **cmd/server**: Ponto de entrada da aplicação. Responsável por inicializar e conectar todos os demais componentes, incluindo o banco de dados, o barramento de eventos, o agendador de tarefas e o servidor HTTP.
- **internal/api**: Camada HTTP da aplicação, implementada com o framework Gin [Gin-Gonic Contributors, 2024]. Divide-se em dois grupos de rotas: endpoints públicos, acessíveis pelas instâncias em `/configs/*`, que entregam os dados de metadados, user-data, vendor-data e network-config; e endpoints internos em `/internal/*`, destinados a operações de gerenciamento.

- **internal/events**: Módulo de sincronização orientada a eventos, utilizando o GoEventBus. Implementa um modelo de publicação e assinatura (*pub/sub*) que desacopla o agendamento da execução da sincronização com o Incus.
- **internal/storage**: Camada de persistência baseada em SQLite [Hipp, 2024], com queries geradas de forma estática e com segurança de tipos pelo SQLC. Armazena os registros de instâncias, seus metadados cloud-init, user-data, vendor-data e configurações de rede.
- **internal/consensus**: Módulo opcional de consenso distribuído baseado na biblioteca `hashicorp/raft`, detalhado na Seção 3.6. Quando habilitado, as operações de escrita são submetidas ao *log* replicado antes de serem aplicadas ao banco de dados local de cada nó.
- **internal/telemetry**: Instrumentação de métricas com OpenTelemetry, exportadas no formato Prometheus pelo endpoint `/metrics`.
- **pkg/types**: Definições de tipos de dados utilizados em toda a aplicação, garantindo compatibilidade com as estruturas esperadas pelo cloud-init.

Na Figura 1, a escrita direta do módulo de eventos na camada de armazenamento corresponde ao modo de nó único; com o consenso habilitado, as escritas seguem o caminho pelo módulo de consenso.

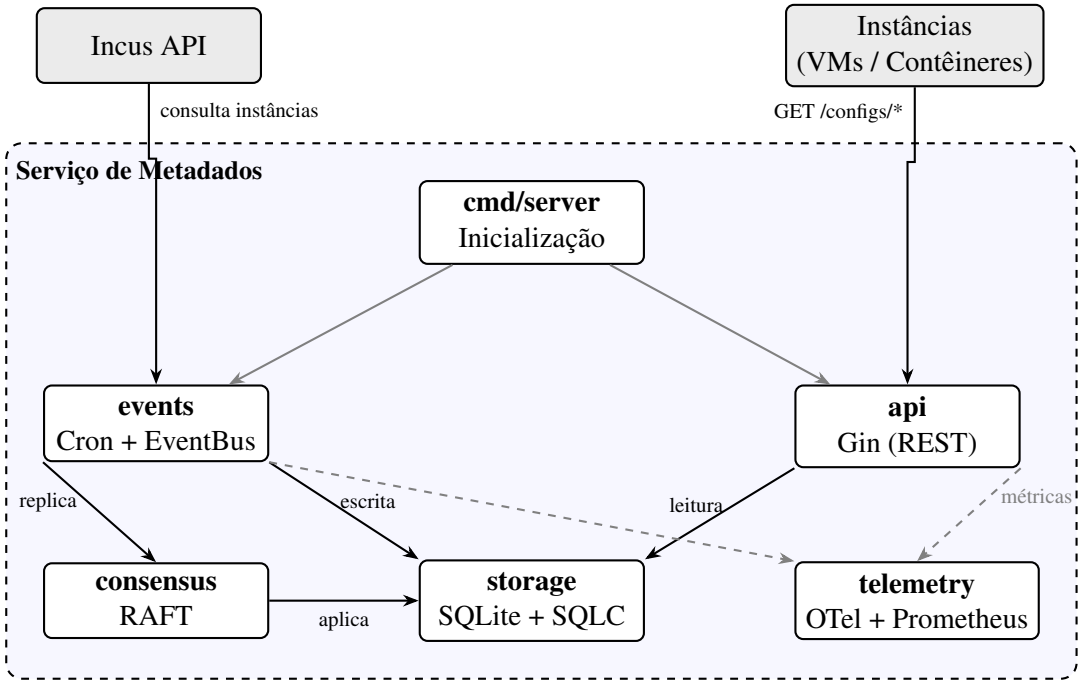


Figura 1. Arquitetura do serviço de metadados e interação entre seus componentes. Setas contínuas indicam fluxos de dados; setas tracejadas indicam instrumentação de métricas.

3.2. Sincronização de Instâncias

Para que o serviço consiga responder corretamente às requisições das instâncias, é necessário manter um estado atualizado de cada instância gerenciada pelo Incus. Isso é realizado por meio de um ciclo de sincronização periódico e orientado a eventos, descrito a seguir:

1. Um agendador de tarefas (*cron*) é executado a cada 10 segundos e publica um evento `instances.sync` no barramento de eventos.
2. O handler do evento `instances.sync` consulta a API do Incus e obtém a lista completa de instâncias ativas no momento.
3. Para cada instância retornada, um evento individual `instance.sync` é publicado, permitindo que o processamento ocorra de forma desacoplada e paralela.
4. O handler do evento `instance.sync` executa as seguintes etapas para cada instância:
    - Extrai informações de rede do estado da instância reportado pelo Incus, incluindo endereços IPv4, IPv6 e endereço MAC de cada interface;
    - Cria ou atualiza o registro da instância no banco de dados;
    - Constrói e persiste os metadados cloud-init, incluindo `instance-id`, hostname, endereços IP, chaves SSH públicas, informações de posicionamento (*placement*) e interfaces de rede. As chaves SSH são extraídas das chaves de configuração `user.meta-data` (campo `public-keys` em YAML) e `user.ssh-keys` (lista separada por quebras de linha);
    - Armazena o user-data a partir da chave de configuração `cloud-init.user-data` presente na configuração da instância no Incus;
    - Armazena o vendor-data a partir da chave de configuração `cloud-init.vendor-data` do Incus, permitindo que perfis e configurações específicas do provedor sejam entregues por instância;
    - Constrói a configuração de rede: caso a chave `cloud-init.network-config` esteja definida na configuração da instância no Incus, utiliza-a diretamente, permitindo ao administrador especificar configurações de rede estáticas, *bonds*, VLANs ou DNS personalizado; caso contrário, gera automaticamente a configuração no formato *Networking Config Version 2* [Canonical Ltd., 2024e], baseada na especificação Netplan, a partir das interfaces reais reportadas pelo estado da instância;
    - Atualiza o estado da instância no banco de dados, incluindo status textual e código de status numérico.

Esse modelo orientado a eventos mantém o estado armazenado consistente com o estado real das instâncias no Incus, dentro da janela do ciclo de sincronização, sem impor acoplamento direto entre o agendamento e a lógica de sincronização.

3.3. Endpoints da API

Os endpoints públicos do serviço seguem o modelo do datasource *NoCloud* HTTP do cloud-init [Canonical Ltd., 2024f], permitindo que instâncias consultem seus próprios dados de configuração a partir do endereço IP de origem da requisição. Os endpoints disponíveis são:

- `GET /configs/meta-data`: Retorna os metadados da instância no formato JSON ou YAML, incluindo `instance-id`, hostname, endereços IP, chaves SSH públicas (`public-keys`) e informações de rede. A instância é identificada pelo seu endereço IP de origem.

- `GET /configs/meta-data/:key`: Retorna um campo específico dos metadados da instância, identificado pelo parâmetro `key`.
- `GET /configs/user-data`: Retorna o user-data associado à instância, conforme configurado na chave `cloud-init.user-data` do Incus. O user-data tipicamente contém scripts ou configurações no formato cloud-config.
- `GET /configs/vendor-data`: Retorna dados de configuração específicos do fornecedor (*vendor-data*), que complementam o user-data sem sobrescrevê-lo. O serviço prioriza o vendor-data por instância, extraído da chave `cloud-init.vendor-data` do Incus; caso não exista, retorna o vendor-data global configurado pela API de gerenciamento.
- `GET /configs/network-config`: Retorna a configuração de rede da instância. Quando a chave `cloud-init.network-config` está definida na configuração da instância no Incus, o conteúdo definido pelo administrador é servido diretamente; caso contrário, uma configuração no formato *Networking Config Version 2* [Canonical Ltd., 2024e] é gerada automaticamente a partir das interfaces reais reportadas pelo Incus. Esse *endpoint* não é consultado pelo *datasource* NoCloud via HTTP (Seção 3.4).

Além dos endpoints públicos, o serviço disponibiliza endpoints internos no prefixo `/internal/` para operações de gerenciamento de vendor-data, permitindo criar, consultar e atualizar registros identificados por nome de fornecedor (`/internal/vendor/:vendor_name/data`). Esses endpoints são destinados ao uso administrativo e não são expostos diretamente às instâncias.

3.4. Descoberta do Serviço pelas Instâncias

O serviço adota o *datasource NoCloud* do cloud-init em modo de rede [Canonical Ltd., 2024f]: o parâmetro `seedfrom` aponta para o prefixo `/configs/` do serviço e, durante a inicialização, o cloud-init concatena esse prefixo com os caminhos `meta-data`, `user-data` e `vendor-data` para compor as URLs de consulta. O caminho `network-config` não é requisitado pelo NoCloud via `seedfrom`, como mostrou a avaliação experimental (Seção 4.2): o *endpoint* correspondente é servido pelo serviço, mas a configuração de rede efetivamente aplicada pelo cloud-init provém do *seed* local gravado pelo Incus. Como a configuração do *datasource* precisa estar disponível antes de o cloud-init obter qualquer dado, o `seedfrom` não pode ser entregue pelo próprio serviço. Ele é definido em um arquivo de configuração do cloud-init (`/etc/cloud/cloud.cfg.d/90-nocloud-net.cfg`) gravado em uma imagem-base publicada no Incus, a partir da qual as instâncias são criadas. Trata-se de uma configuração feita uma única vez, na imagem, e não por instância; nenhum agente adicional é necessário dentro das instâncias.

O endereço para o qual o `seedfrom` aponta admite duas formas. A primeira reproduz a convenção dos provedores de nuvem pública, o endereço *link-local* `169.254.169.254` [Amazon Web Services, 2024a; Google Cloud, 2024]: esse endereço é adicionado à interface de ponte gerenciada pelo Incus (`incusbr0`) e, como o cloud-init realiza requisições na porta 80 por padrão, uma regra de redirecionamento NAT encaminha o tráfego destinado a `169.254.169.254:80` para a porta em que o serviço está em execução. Essa forma é adequada a hosts dedicados, como em *homelabs* e servidores *bare metal*. A segunda forma usa diretamente o endereço de *gateway* da ponte do Incus e a

porta do serviço, sem NAT, e é necessária quando o próprio host é uma máquina virtual de um provedor de nuvem, no qual o endereço `169.254.169.254` já está em uso pelo serviço de metadados do provedor. A avaliação experimental (Seção 4) utilizou a segunda forma.

3.5. Tecnologias Utilizadas

A escolha das tecnologias que compõem o serviço foi guiada por critérios de desempenho, simplicidade operacional e adequação ao problema:

- **Go**: A linguagem oferece desempenho próximo ao de linguagens de mais baixo nível, como C, com um modelo de concorrência baseado em goroutines que facilita o tratamento paralelo de eventos. Além disso, o processo de compilação gera um único binário estático, simplificando o processo de implantação em ambientes de borda.
- **Gin** [Gin-Gonic Contributors, 2024]: Framework HTTP de alta performance para Go, com roteamento eficiente e baixo overhead. Sua API expressiva permite definir grupos de rotas e middlewares de forma clara, facilitando a separação entre os endpoints públicos e internos.
- **SQLite + SQLC** [Hipp, 2024]: O SQLite é um banco de dados relacional embutido, sem necessidade de processo servidor separado, adequado para serviços com requisitos de implantação simplificados. O SQLC complementa essa escolha ao gerar código Go com tipagem estática a partir de queries SQL declaradas, eliminando o uso de ORMs genéricos e reduzindo a superfície de erros em tempo de execução.
- **GoEventBus**: Biblioteca de barramento de eventos para Go que implementa o padrão publicação/assinatura. Permite desacoplar o agendamento da sincronização da lógica de processamento por instância, tornando o sistema mais modular e testável.
- **gocron**: Biblioteca de agendamento de tarefas periódicas para Go, utilizada para disparar o ciclo de sincronização a cada 10 segundos de forma confiável e configurável.
- **hashicorp/raft**: Biblioteca de consenso distribuído para Go que implementa o protocolo RAFT [Ongaro and Ousterhout, 2014]. Utilizada para coordenar escritas em implantações com múltiplos nós, garantindo que apenas o líder eleito realize sincronizações com o Incus e proponha alterações ao cluster.
- **OpenTelemetry + Prometheus** [Cloud Native Computing Foundation, 2024a,b]: O serviço utiliza o OpenTelemetry como framework de instrumentação para coleta de métricas, seguindo o padrão aberto da Cloud Native Computing Foundation (CNCF). As métricas são exportadas no formato Prometheus por meio de um endpoint `/metrics`, permitindo o monitoramento do serviço por ferramentas de observabilidade padrão do ecossistema nativo de nuvem. As métricas instrumentadas incluem a duração do processamento de eventos de sincronização (histograma `event_processing_duration_seconds`) e a contagem total de eventos processados (`event_processing_total`), ambas segmentadas por tipo de projeção e status de execução.

3.6. Consenso Distribuído via RAFT

A arquitetura de nó único apresenta um ponto único de falha (*Single Point of Failure*, SPOF): caso o nó que executa o serviço de metadados fique indisponível, todas as instâncias perdem acesso às suas configurações de inicialização, o que inviabiliza reinicializações e provisionamentos durante o período de falha. Para possibilitar implantações de alta disponibilidade em ambientes de produção, o serviço implementa consenso distribuído por meio do protocolo RAFT [Ongaro and Ousterhout, 2014], utilizando a biblioteca `hashicorp/raft`.

O suporte a RAFT é ativado de forma opcional por meio de variáveis de ambiente, sendo `RAFT_ENABLED=true` o ponto de entrada. Quando habilitado, o nó é configurado com os parâmetros definidos nas variáveis `NODE_ID`, `BIND_ADDR`, `DATA_DIR`, `PEERS` e `BOOTSTRAP`, permitindo compor um cluster sem alterações no binário.

O funcionamento do módulo de consenso é organizado em três camadas principais:

- **Eleição de líder**: O protocolo RAFT garante que, a qualquer momento, no máximo um nó detenha o papel de líder. Apenas o nó líder executa o ciclo de sincronização com o Incus e propõe comandos de escrita ao cluster. Os nós seguidores não realizam sincronizações independentes; caso o líder falhe, uma nova eleição é conduzida automaticamente e o nó eleito passa a assumir as responsabilidades de sincronização.
- **Replicação de log e FSM**: Cada operação de escrita (criação, atualização e remoção de instância; persistência de metadados, user-data, vendor-data, network-config e estado) é codificada como um comando tipado (`Command`) e submetida ao log replicado via `hashicorp/raft`. Após o *commit* do log em uma maioria de nós, a *Finite State Machine* (FSM) aplica o comando ao banco de dados SQLite local de cada nó, mantendo o estado consistente em todo o cluster. Os tipos de comando suportados são: `CmdCreateInstance`, `CmdUpdateInstance`, `CmdCreateOrUpdateMetadata`, `CmdCreateOrUpdateUserData`, `CmdCreateOrUpdateNetworkConfig`, `CmdCreateOrUpdateState`, `CmdCreateOrUpdateVendorData` e `CmdDeleteInstance`, este último utilizado pela rotina de reconciliação que remove registros de instâncias não mais presentes no Incus.
- **Armazenamento persistente**: O log de entradas RAFT e o estado estável do nó são persistidos em um arquivo BoltDB localizado no diretório configurado por `DATA_DIR`. Essa escolha elimina a necessidade de um processo de banco de dados externo para o próprio mecanismo de consenso, mantendo o requisito de implantação simplificada.

Todos os nós do cluster, tanto o líder quanto os seguidores, podem responder a requisições de leitura, uma vez que cada um mantém uma cópia local do estado no SQLite. Essas leituras são eventualmente consistentes: um seguidor pode estar defasado em relação ao líder pelo intervalo entre o *commit* de uma entrada no *log* e sua aplicação local pela FSM. Como as escritas ocorrem apenas no ciclo de sincronização periódica, essa janela é aceitável para o perfil do serviço e permite distribuir a carga de consultas de metadados entre os nós.

4. Resultados

Esta seção apresenta a avaliação do serviço de metadados proposto. A avaliação é organizada em seis partes: a metodologia experimental adotada, os testes funcionais que verificam a corretude do provisionamento, os testes de desempenho que quantificam a latência e a escalabilidade do serviço, a avaliação de alta disponibilidade baseada em consenso RAFT, uma comparação de funcionalidades com serviços de metadados de provedores de nuvem públicos e a discussão dos resultados e de suas limitações.

De modo geral, os experimentos confirmam a compatibilidade funcional do serviço com o *cloud-init*: instâncias reais selecionam o *datasource* de rede, obtêm seus metadados e concluem a inicialização com sucesso. As latências de resposta situam-se na faixa de dezenas de milissegundos e, ao contrário da hipótese inicial de degradação por serialização de escrita do SQLite, o serviço manteve latência estável e ausência de erros até 200 contêineres de sistema simultâneos em um único host, sob carga HTTP constante. A avaliação de alta disponibilidade demonstrou formação de *cluster*, replicação de estado e reeleição de líder da ordem de segundos (média de 2,04 s em dez repetições). Os detalhes de cada dimensão são apresentados a seguir.

4.1. Metodologia de Avaliação

Os experimentos foram conduzidos em uma instância de nuvem Google Compute Engine do tipo `e2-standard-4`, com 4 vCPUs (processador Intel Xeon a 2,20 GHz), 15 GiB de memória RAM, executando Ubuntu 24.04 LTS (*kernel* 6.17.0-1020-gcp) e Incus 6.0.0. O serviço de metadados foi implantado no mesmo host das instâncias. A varredura de escalabilidade foi reexecutada posteriormente em um host do mesmo tipo, com *kernel* 7.0.0-1011-gcp e disco de 250 GB, por ser este o requisito de armazenamento do *pool* `dir` do Incus com 200 contêineres; os valores reportados na Seção 4.3 provêm dessa reexecução. A avaliação de alta disponibilidade utilizou um segundo ambiente, composto por três máquinas virtuais `e2-standard-2` (2 vCPUs e 8 GiB de RAM) na mesma rede interna, cada uma executando uma réplica do serviço; a topologia adotada é descrita na Seção 4.4.

Um aspecto prático relevante emergiu durante a configuração do ambiente: em provedores de nuvem pública (Google Cloud, AWS e Azure), o endereço `169.254.169.254` já é utilizado pelo próprio serviço de metadados do provedor, de modo que sua reutilização no host interfere na resolução de nomes e no acesso aos metadados da própria máquina virtual. Por essa razão, no ambiente de testes o *cloud-init* foi apontado, via *datasource* NoCloud, para o endereço de *gateway* da *bridge* do Incus (`10.10.10.1:8080`), diretamente alcançável pelas instâncias, conforme a segunda forma de endereçamento descrita na Seção 3.4. Esse achado tem relevância prática por si só: a convenção do endereço reservado, embora adequada para instâncias hospedadas dentro de um provedor, não é diretamente transponível para um host que já opera sob tal provedor.

O parâmetro `seedfrom` foi gravado em uma imagem-base Ubuntu 24.04 publicada no Incus, a partir da qual todos os contêineres foram criados. Nessa imagem, a gestão de rede pelo *cloud-init* foi desabilitada (`network: {config: disabled}`) e a interface `eth0` foi configurada por DHCP via Netplan, de modo a evitar um impasse de inicialização: sem rede, o contêiner não alcançaria o serviço que lhe forneceria a própria

configuração de rede. Como consequência, nos experimentos principais o conteúdo do *endpoint* `/network-config` foi servido e inspecionado, mas não aplicado pelo *cloud-init*. Um experimento complementar, descrito na Seção 4.2, utilizou uma variante dessa imagem com a gestão de rede do *cloud-init* habilitada para verificar o consumo desse *endpoint*.

Para avaliar o comportamento sob diferentes cargas, foram criados grupos de instâncias com cardinalidades crescentes, de 5 até 200 contêineres simultâneos. Cada instância foi configurada com o *cloud-init* habilitado, apontando para o serviço de metadados proposto como fonte de configuração. Durante a inicialização, o *cloud-init* realizou requisições aos *endpoints* `/meta-data`, `/user-data` e `/vendor-data` do serviço, seguindo o fluxo de *datasource* documentado em [Canonical Ltd., 2024a]; o *endpoint* `/network-config`, embora implementado, não é consultado pelo *datasource* NoCloud via HTTP (Seção 4.2).

As medições foram realizadas em três dimensões: (a) **latência de resposta da API**, correspondente ao tempo decorrido entre o envio da requisição HTTP pela instância e o recebimento da resposta; (b) **acurácia de sincronização**, verificando se os metadados armazenados refletem o estado atual das instâncias conforme reportado pelo Incus; e (c) **corretude da configuração**, avaliando se as configurações entregues foram efetivamente aplicadas pelas instâncias ao término da inicialização, com exceção da configuração de rede, pelo motivo exposto acima.

A coleta de latência foi realizada com a ferramenta `hey`, disparando 5000 requisições por configuração com concorrência de 50 requisições simultâneas a partir de um contêiner gerador de carga. Essa carga HTTP é idêntica em todas as configurações; o que varia entre elas é o número de contêineres em execução no host, que determina o volume de escritas realizado a cada ciclo de sincronização e o consumo de recursos do host. O banco SQLite foi aberto em modo WAL (*Write-Ahead Logging*), com *busy timeout* de 5 s e uma única conexão aberta (`SetMaxOpenConns(1)`), o que serializa todo o acesso ao banco dentro do processo do serviço. Os resultados de latência são reportados nos percentis p50, p95 e p99, seguindo a prática adotada em sistemas de produção para caracterizar tanto o comportamento típico quanto os casos de cauda da distribuição.

4.2. Avaliação Funcional

A avaliação funcional verificou se o serviço de metadados entrega as informações corretas a cada instância e se essas informações são interpretadas e aplicadas com êxito pelo *cloud-init*. As propriedades testadas e seus resultados são apresentados na Tabela 2.

1. **Isolamento de metadados**: cada instância deve receber exclusivamente os metadados associados ao seu próprio identificador. Múltiplas instâncias realizaram requisições simultâneas ao serviço, e confirmou-se que o campo `local-hostname` correspondia unicamente à instância solicitante.
2. **Isolamento sob manipulação do cabeçalho `X-Forwarded-For`**: como o serviço identifica a instância pelo endereço IP de origem da requisição, foi testada uma tentativa de forjar essa identidade por meio do cabeçalho HTTP `X-Forwarded-For`, preenchido com o IP de outra instância. O IP de origem da conexão não foi alterado; apenas o cabeçalho. Confirmou-se que a requisição

não retornou os dados da outra instância, pois o serviço ignora esse cabeçalho ao desativar a confiança em *proxies* (`SetTrustedProxies(nil)`).

3. **Execução de *user-data***: um *script* de *user-data* injetado via chave `cloud-init.user-data` do Incus foi executado durante a inicialização, criando um arquivo sentinela verificado após o término do *cloud-init*.
4. **Entrega de configuração de rede**: a configuração entregue pelo *endpoint* `/network-config` foi servida como YAML válido, com as interfaces reais reportadas pelo Incus, e capturada para comparação com a saída de `ip addr` na instância. Como a imagem-base desabilita a gestão de rede do *cloud-init* (Seção 4.1), a aplicação foi verificada em um experimento complementar com uma variante da imagem em que essa gestão foi reabilitada, mantendo-se um DHCP de inicialização por nome de interface. Nesse experimento, o *cloud-init* (versão 26.1) concluiu com `status: done` usando o *datasource* NoCloud de rede, mas o registro de acessos do serviço mostrou requisições apenas a `/meta-data`, `/user-data` e `/vendor-data`: o *datasource* NoCloud não requisita `network-config` pela URL `seedfrom`, o que foi confirmado na função `read_seeded` do código-fonte da ferramenta. A configuração de rede efetivamente aplicada pelo *cloud-init* proveio do *seed* local que o próprio Incus grava em `/var/lib/cloud/seed/nocloud-net/` a partir da chave `cloud-init.network-config`. Portanto, o *endpoint* `/network-config` do serviço é servido corretamente, mas não é consumido pelo *cloud-init* no fluxo NoCloud via HTTP.
5. **Atualização após mudança de estado**: após a reinicialização de uma instância, os metadados persistidos refletiram o novo estado dentro da janela de sincronização periódica.

Tabela 2. Resultados dos testes funcionais.

Propriedade	**Resultado**
Isolamento de metadados	Aprovado
Isolamento sob manipulação do cabeçalho `X-Forwarded-For`	Aprovado
Execução de *user-data*	Aprovado
Entrega de *network-config*	Conteúdo servido; não consultado pelo NoCloud via HTTP (não aplicado pelo *cloud-init*)
Propagação de estado após reinício	Aprovado

As propriedades de isolamento, execução de *user-data* e propagação de estado foram atendidas. A entrega de *network-config* foi validada quanto ao conteúdo servido, mas o experimento complementar mostrou que esse *endpoint* não é consumido pelo *datasource* NoCloud via HTTP na versão de *cloud-init* avaliada. Em particular, uma inicialização real de *cloud-init* selecionou o *datasource* `NoCloud` de rede, obteve seus metadados e *user-data* do serviço (respostas HTTP 200) e concluiu com `cloud-init status: done`, evidenciando a compatibilidade funcional de ponta a ponta para *meta-data* e *user-data* em contêineres de sistema.

4.3. Avaliação de Desempenho

A avaliação de desempenho concentrou-se em três métricas: latência de resposta da API, latência de sincronização e escalabilidade sob variação do número de instâncias.

4.3.1. Latência de Resposta da API

A latência foi medida para os quatro *endpoints* do serviço. A Tabela 3 apresenta os percentis p50, p95 e p99 obtidos com 5000 requisições por *endpoint*. O *endpoint* `/vendor-data` respondeu com código HTTP 404 por não haver dado de *vendor* configurado no cenário (comportamento correto, tolerado pelo *cloud-init*), mas suas latências permanecem representativas do custo de processamento da requisição.

Tabela 3. Latência de resposta por *endpoint* (5000 requisições, concorrência 50).

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)
`/meta-data`	13,2	53,7	83,2
`/user-data`	18,9	53,6	72,4
`/network-config`	11,4	47,2	72,4
`/vendor-data`	17,0	48,4	68,9

As latências medianas situam-se em torno de 11–19 ms e o percentil p99 abaixo de 90 ms para todos os *endpoints*, valores compatíveis com as janelas de tempo disponíveis durante a inicialização de instâncias.

4.3.2. Latência de Sincronização

A latência de sincronização (intervalo entre a criação de uma instância no Incus e a disponibilização de seu registro de metadados) foi medida, em uma única execução, em aproximadamente **20,5 s**. Esse valor é consistente com a janela do ciclo de sincronização periódica adotado (*polling* a cada 10 s) somada ao tempo de inicialização da instância e de obtenção de endereço de rede.

4.3.3. Escalabilidade

Para avaliar o comportamento com o aumento do número de instâncias simultâneas, mediu-se a latência de resposta do *endpoint* `/meta-data` para configurações de 5 a 200 contêineres de sistema em execução no mesmo host. A variável independente deste experimento é apenas o número de contêineres em execução: a carga HTTP é a mesma em todas as configurações (5000 requisições com concorrência fixa de 50, disparadas por um único contêiner gerador de carga). O experimento mede, portanto, o efeito de mais instâncias sincronizadas e de maior ocupação do host sobre a latência; o número de clientes HTTP simultâneos não cresce com o número de contêineres. A hipótese inicial era de que, por utilizar SQLite como mecanismo de persistência, o serviço apresentasse degradação de desempenho sob alta concorrência de escrita, dado que o SQLite serializa operações de escrita [Hipp, 2024]. A Tabela 4 apresenta os resultados e a Figura 2 os representa

graficamente. Cada configuração corresponde a uma única execução de 5000 requisições, de modo que os valores devem ser lidos como indicativos.

Tabela 4. Escalabilidade: latência de `/meta-data` e memória utilizada no host por número de contêineres em execução. A carga HTTP é constante em todas as linhas (5000 requisições, concorrência 50).

Instâncias	p50 (ms)	p95 (ms)	p99 (ms)	Erros	Mem. host (MB)
5	19,3	77,7	117,4	0,0%	1216
10	14,5	58,0	87,6	0,0%	1404
25	12,7	52,5	78,5	0,0%	2053
50	12,6	53,5	78,5	0,0%	3147
100	14,3	60,0	94,9	0,0%	4715
150	14,9	69,1	124,8	0,0%	4909
200	13,2	55,6	85,4	0,0%	5171

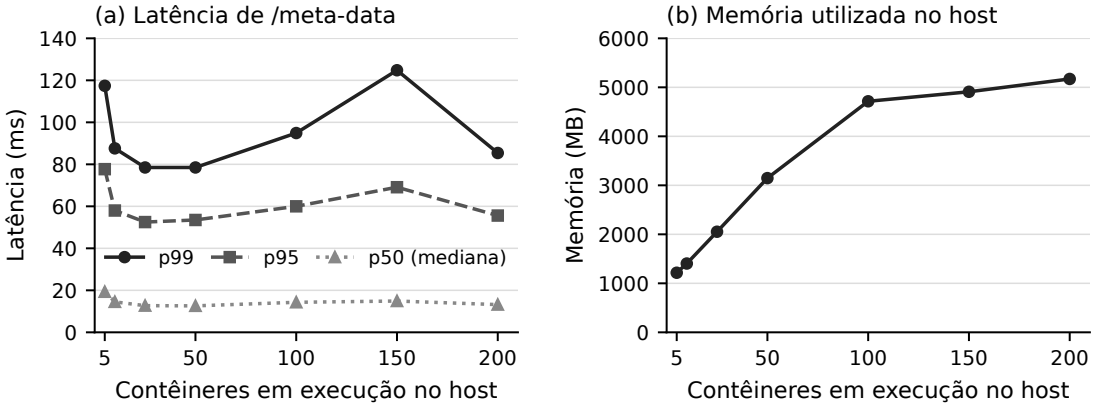


Figura 2. Escalabilidade em host único com contêineres de sistema, em função do número de contêineres em execução: (a) latência de `/meta-data` nos percentis p50 (mediana), p95 e p99; (b) memória utilizada no host. A carga HTTP é a mesma em todos os pontos (uma execução de 5000 requisições com concorrência fixa de 50); apenas o número de contêineres varia. Valores indicativos.

Os dados **não corroboram** a hipótese de degradação por serialização de escrita nessa faixa de escala: a latência não apresentou tendência de crescimento com o número de contêineres (p50 entre 12,6 e 19,3 ms; p99 entre 78,5 e 124,8 ms, com o maior valor em 150 contêineres e o menor em 25 e 50) e a taxa de erro foi nula em todas as configurações, até 200 contêineres de sistema em um único host de 4 vCPUs, sob carga HTTP constante de 5000 requisições com concorrência 50. Como a concorrência HTTP não acompanhou o número de contêineres, o resultado não indica o comportamento do serviço quando muitas instâncias o consultam ao mesmo tempo, como em uma inicialização simultânea em massa. Esse resultado também não foi medido com máquinas virtuais nem em topologias com múltiplos hosts. Esse resultado decorre da natureza do fluxo predominante, majoritariamente requisições de leitura (`GET`) de metadados, enquanto as escritas de sincronização ocorrem apenas a cada 10 s e são absorvidas pelo modo WAL e pelo *busy timeout* configurados no SQLite (Seção 4.1). Conclui-se que a serialização de escrita não é um gargalo no perfil de carga avaliado; uma eventual degradação exigiria contagens de instâncias ou

taxas de escrita substancialmente superiores, o que se registra como limite medido e não como tendência de degradação linear.

4.4. Avaliação de Alta Disponibilidade

Para avaliar o mecanismo de alta disponibilidade baseado em consenso RAFT, provisionou-se um *cluster* de três nós (`e2-standard-2`) com endereços internos fixos, um dos quais designado como nó de inicialização (*bootstrap*). Adotou-se a topologia de **réplicas sobre um Incus compartilhado**: os três serviços de metadados apontam para a mesma instância do Incus, de modo que o líder sincroniza o estado a partir de uma visão única de instâncias e o replica aos seguidores via *log* do RAFT. A Tabela 5 resume os resultados.

Tabela 5. Resultados da avaliação de alta disponibilidade (*cluster* de 3 nós).

Propriedade	**Resultado**
Formação do *cluster*	1 líder + 2 seguidores
Replicação de estado	instância criada no líder presente nos 3 nós
Reeleição após falha do líder	2,04 s em média (10 execuções; mín. 1,46 s, máx. 2,65 s)
Disponibilidade dos dados após *failover*	preservada (sem remoção indevida)
Reintegração de nó	nó reiniciado reingressa como seguidor

O *cluster* formou-se corretamente, com eleição de um único líder. Uma instância criada no Incus compartilhado teve seu registro replicado aos três nós. Ao encerrar-se o serviço do líder (mantendo-se o Incus compartilhado ativo), um novo líder foi eleito em 2,40 s em uma primeira execução do cenário, e os dados replicados permaneceram disponíveis nos nós sobreviventes, sem remoção indevida. Um nó previamente encerrado foi capaz de reingressar ao *cluster* e recuperar o estado a partir do *log*. Para caracterizar a variabilidade do tempo de reeleição, o cenário de falha foi repetido dez vezes em um *cluster* reprovisionado com a mesma configuração: a cada repetição, o processo do líder corrente foi encerrado com `SIGKILL` e um nó sobrevivente mediu, por sondagem a cada 50 ms pela rede interna, o intervalo entre a última resposta do líder e a primeira resposta de um nó como novo líder. Todas as repetições resultaram em reeleição, com tempo **médio de 2,04 s** (mediana 2,04 s; mínimo 1,46 s; máximo 2,65 s; desvio padrão 0,38 s), valores coerentes com os *timeouts* configurados (*heartbeat* de 1 s e eleição de 1 s, com temporização aleatória do RAFT). Nessas repetições mediu-se apenas o tempo de reeleição; a disponibilidade dos dados após o *failover* foi verificada na execução inicial.

O tempo de reeleição não equivale ao tempo de indisponibilidade do serviço percebido pelas instâncias, que não foi medido. Durante a eleição, os seguidores sobreviventes continuam aptos a responder leituras a partir de sua cópia local do estado (Seção 3.6); o que fica suspenso é a sincronização com o Incus e a replicação de novas escritas, retomadas pelo novo líder no ciclo de sincronização seguinte (até 10 s após a eleição). Para as instâncias, a indisponibilidade efetiva depende de como alcançam o *cluster*: uma instância cujo `seedfrom` aponte para o endereço do nó que falhou permanece sem resposta até que esse nó retorne, independentemente da reeleição, e nenhum mecanismo de endereço virtual ou balanceamento entre os nós foi avaliado neste trabalho.

Além disso, a topologia de Incus compartilhado torna o serviço dependente de uma única instância do Incus: a replicação via RAFT protege contra a falha do processo do serviço de metadados, mas não contra a falha do próprio Incus. Se o Incus compartilhado ficar indisponível, nenhum nó consegue sincronizar, e o *cluster* passa a servir apenas o último estado replicado até que o Incus volte. Esse cenário não foi testado.

Observou-se que a corretude da rotina de reconciliação (que remove registros de instâncias ausentes do Incus) depende da topologia adotada: sob a topologia de Incus compartilhado, todo nó que possa assumir a liderança observa o mesmo conjunto de instâncias, de modo que a reconciliação é correta e o *failover* não provoca perda de dados. Uma topologia alternativa, com um Incus por nó (adequada a cenários de borda distribuída), exigiria reconciliação restrita às instâncias de cada nó de origem, de forma que um nó jamais removesse instâncias pertencentes a outro.

4.5. Comparação com Provedores de Nuvem

Para contextualizar as capacidades do serviço proposto, a Tabela 6 compara os campos de metadados disponibilizados com os oferecidos pelo AWS Instance Metadata Service [Amazon Web Services, 2024a], pelo serviço de metadados do Google Cloud Platform [Google Cloud, 2024] e pelo serviço de metadados do OpenStack [OpenStack Foundation, 2024].

Tabela 6. Comparação de campos de metadados (✓ suportado; ✗ não suportado; P: parcial). Para o serviço proposto, `network-config` é marcado como parcial: o conteúdo é servido, mas não é consumido pelo *datasource* NoCloud via HTTP.

Campo	Proposto	AWS	GCP	OpenStack
`instance-id`	✓	✓	✓	✓
`local-hostname`	✓	✓	✓	✓
`local-ipv4`	✓	✓	✓	✓
Chaves SSH públicas	✓	✓	✓	✓
`availability-zone`	✓	✓	✓	P
Tipo de instância	✗	✓	✓	✓
`user-data`	✓	✓	✓	✓
`vendor-data`	✓	✗	✗	✓
`network-config`	P	P	P	✓

O serviço proposto cobre os campos essenciais para a operação dos módulos mais comuns do *cloud-init* (`set_hostname`, `users_groups`, `write_files` e `runcmd`), oferecendo também entrega de *vendor-data* por instância com *fallback* global. A configuração de rede é coberta apenas parcialmente: o conteúdo é servido, mas o *datasource* NoCloud do *cloud-init* não consome esse *endpoint* via HTTP (Seção 4.2), e as correções feitas posteriormente na geração automática dessa configuração não foram reavaliadas experimentalmente. Campos específicos de provedores públicos, como tipo de instância e atributos de faturamento, não são expostos, por não se aplicarem ao contexto de *homelab* e computação de borda visado.

4.6. Discussão

Os resultados obtidos permitem avaliar em que medida o serviço de metadados proposto atende aos requisitos de um ambiente Incus de pequena escala. Do ponto de vista funcio-

nal, o serviço demonstrou corretude no isolamento de metadados (inclusive sob manipulação do cabeçalho `X-Forwarded-For`) e na entrega de *meta-data* e *user-data* efetivamente aplicados pelo *cloud-init*, validando a premissa de compatibilidade com o *datasource* de rede [Canonical Ltd., 2024a]; a configuração de rede servida, contudo, não é consumida pelo *datasource* NoCloud via HTTP. Do ponto de vista de desempenho, a natureza leve da implementação traduziu-se em latências de dezenas de milissegundos e em latência estável até 200 contêineres de sistema em host único. Do ponto de vista de disponibilidade, o mecanismo de consenso demonstrou reeleição de líder da ordem de segundos, de forma consistente em dez repetições do cenário de falha; esse valor mede o intervalo sem líder; a indisponibilidade efetiva do serviço para as instâncias não foi medida.

Entre as limitações do serviço proposto, destacam-se:

- **Escalabilidade do SQLite**: embora não se tenha observado degradação até 200 contêineres em host único, o SQLite serializa operações de escrita, o que poderia tornar-se um gargalo sob taxas de criação e destruição de instâncias muito superiores às avaliadas. Essa característica é inerente à escolha por uma solução sem dependências externas e é aceitável no perfil de carga de *homelab* e computação de borda de pequena escala.
- **Cobertura parcial de campos de metadados**: o serviço implementa o subconjunto de campos necessários aos módulos mais comuns do *cloud-init*, mas não cobre todos os campos de provedores como AWS e GCP [Amazon Web Services, 2024a; Google Cloud, 2024]. Cargas que dependam de campos específicos podem requerer extensões.
- **Dependência da topologia na alta disponibilidade**: as garantias de disponibilidade sem perda de dados no *failover* pressupõem a topologia de réplicas sobre um Incus compartilhado; cenários com um Incus por nó demandam reconciliação por nó de origem. Nessa topologia, o Incus compartilhado permanece como ponto único de falha para a sincronização, e o acesso das instâncias a um nó sobrevivente depende de um mecanismo de endereçamento (endereço virtual ou balanceador) não avaliado.
- **Configuração de rede não consumida via HTTP**: o *datasource* NoCloud do *cloud-init* lê `network-config` apenas de fontes locais, de modo que o *endpoint* `/network-config` do serviço só é útil a consumidores que o consultem diretamente. Integrar a configuração de rede ao fluxo do *cloud-init* exigiria um *datasource* que a obtenha pela rede ou a gravação do *seed* local do Incus. Além disso, na versão avaliada, a configuração gerada automaticamente usava casamento por endereço MAC, que o netplan traduz em `PermanentMACAddress` e que interfaces *veth* de contêineres não satisfazem, e omitia gateway, servidores DNS e o prefixo do endereço IPv6. Esses defeitos foram corrigidos no código após os experimentos (interfaces identificadas por nome, gateway e DNS obtidos da rede gerenciada do Incus à qual a interface está ligada, prefixo IPv6 incluído, e DHCP como alternativa quando o gateway é desconhecido), mas a versão corrigida não foi reavaliada e, pelo motivo exposto, não é consumida pelo *cloud-init* no fluxo NoCloud via HTTP.
- **Ausência de autenticação nas requisições**: o serviço não exige cabeçalho nem token, de modo que uma aplicação vulnerável a SSRF dentro de uma instância pode obter os metadados dessa instância pela rede interna, mesmo sem exposição

do serviço à internet (Seção 2). A identificação por IP de origem limita o acesso aos dados da própria instância, mas não elimina esse risco.

Quanto às ameaças à validade, os experimentos de desempenho e escalabilidade foram executados em host único, o que não captura efeitos de rede real entre o host Incus e as instâncias em topologias amplamente distribuídas; a avaliação de alta disponibilidade, contudo, já emprega um *cluster* de três nós. A verificação do consumo de `/network-config` foi realizada com a versão 26.1 do *cloud-init*; versões futuras podem passar a obter esse arquivo pela URL `seedfrom`. A medição de latência de sincronização corresponde a uma execução única e deve ser lida como indicativa, não como estimativa estatística; o tempo de reeleição foi repetido dez vezes, amostra suficiente para caracterizar a ordem de grandeza e a dispersão, mas não para inferência estatística mais fina. As instâncias utilizadas são contêineres do tipo sistema (*system containers*), que compartilham o *kernel* do host, podendo apresentar latências de inicialização distintas das de máquinas virtuais completas; os resultados funcionais e de escalabilidade não devem, portanto, ser estendidos a máquinas virtuais sem nova avaliação. Por fim, a variabilidade do intervalo de sincronização periódica introduz uma janela de inconsistência cujos efeitos dependem da frequência de mudanças de estado no ambiente em produção. Em conjunto, os achados sustentam o objetivo definido na Seção 1: prover, para ambientes Incus, um serviço de metadados compatível com *cloud-init*, leve e com suporte a alta disponibilidade.

5. Conclusão

Este trabalho apresentou o projeto e a implementação de um serviço de metadados compatível com cloud-init para o Incus. O serviço preenche a lacuna identificada na revisão de trabalhos relacionados, oferecendo um endpoint de metadados independente que as instâncias podem consultar para obter suas configurações de inicialização, de forma análoga ao que provedores de nuvem pública disponibilizam nativamente. Foram implementados os endpoints REST exigidos pelas fontes de dados do cloud-init (`meta-data`, `user-data`, `vendor-data` e `network-config`), além de um sistema de sincronização orientado a eventos que mantém os dados das instâncias atualizados no banco de dados. A identificação das instâncias por endereço IP dispensa agentes dentro das instâncias; a única configuração necessária é o parâmetro `seedfrom` do *datasource* NoCloud, definido uma única vez na imagem-base. O serviço implementa ainda suporte a consenso distribuído via RAFT, habilitando implantações de alta disponibilidade com múltiplos nós coordenados por eleição de líder, replicação de log e aplicação determinística de comandos de escrita por meio de uma máquina de estados finita (FSM).

A avaliação experimental verificou a corretude funcional e o desempenho do serviço em cenários com diferentes números de instâncias simultâneas (de 5 a 200 contêineres), bem como o mecanismo de alta disponibilidade baseado em consenso RAFT em um *cluster* de três nós. Os resultados validaram a viabilidade da abordagem em um ambiente Incus real com contêineres de sistema em host único: as inicializações de *cloud-init* completaram-se com sucesso, as latências de resposta situaram-se na faixa de dezenas de milissegundos e a reeleição de líder após falha ocorreu em 2,04 s em média, em dez repetições do cenário (entre 1,46 s e 2,65 s), valor que mede o intervalo sem líder; a indisponibilidade efetiva do serviço para as instâncias não foi medida. A configuração de rede servida pelo *endpoint* `/network-config` não é consumida pelo *datasource* NoCloud via

HTTP na versão de *cloud-init* avaliada, e as correções realizadas posteriormente na geração dessa configuração não foram reavaliadas experimentalmente. Os resultados também não foram medidos com máquinas virtuais. Algumas limitações foram identificadas. O uso do SQLite impõe restrições à concorrência de escrita em cenários de alta carga, ainda que não se tenha observado degradação até 200 contêineres em host único. O conjunto de campos de metadados suportados também representa um subconjunto do que plataformas como AWS e GCP oferecem.

Como trabalhos futuros, destacam-se as seguintes direções: (i) eliminar a dependência de uma imagem-base pré-configurada, por exemplo por meio de um *datasource* nativo do Incus no cloud-init ou da passagem do parâmetro `seedfrom` pela linha de comando do *kernel* em máquinas virtuais; (ii) ampliar o suporte a módulos e campos do cloud-init para maior compatibilidade com cargas de trabalho existentes; (iii) implementar autenticação baseada em sessões inspirada no IMDSv2 [Amazon Web Services, 2024b] para mitigar riscos de SSRF [MITRE Corporation, 2021]; (iv) substituir o mecanismo de sincronização por *polling* pela API de eventos do Incus, obtendo sincronização em tempo real com menor sobrecarga; (v) realizar *benchmarks* de desempenho em cenários de maior escala, com repetições suficientes para análise estatística; e (vi) integrar a configuração de rede a um caminho efetivamente consumido pelo cloud-init, por meio de um *datasource* que a obtenha pela rede ou da gravação do *seed* local do Incus, e reavaliar o conteúdo gerado automaticamente, já corrigido após os experimentos. Estas melhorias consolidariam o serviço como uma solução robusta para ambientes privados que demandam automação de infraestrutura baseada em cloud-init.

Referências

Amazon Web Services (2019). Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 instance metadata service.

Amazon Web Services (2024a). Instance metadata and user data — Amazon EC2 documentation.

Amazon Web Services (2024b). Use IMDSv2 — Amazon EC2 documentation.

Artač, M., Borovšak, T., Di Nitto, E., Guerriero, M., and Tamburri, D. A. (2017). Devops: Introducing infrastructure-as-code. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 497–498. IEEE.

Bollig, E. F., Allan, G. T., Lynch, B. J., Huerta, Y. A., Mix, M., Munsell, E. A., Benson, R. M., and Swartz, B. (2018). Leveraging OpenStack and Ceph for a controlled-access data cloud. In *Proceedings of the Practice and Experience on Advanced Research Computing (PEARC '18)*, Pittsburgh, PA, USA. ACM.

Bresnahan, J., Freeman, T., LaBissoniere, D., and Keahey, K. (2011). Managing appliance launches in infrastructure clouds. In *Proceedings of the 2011 TeraGrid Conference: Extreme Digital Discovery*, Salt Lake City, UT, USA. ACM.

Canonical Ltd. (2024a). cloud-init documentation.

Canonical Ltd. (2024b). EC2 datasource — cloud-init documentation.

Canonical Ltd. (2024c). LXD datasource — cloud-init documentation.

Canonical Ltd. (2024d). LXD documentation.

Canonical Ltd. (2024e). Networking config version 2 — cloud-init documentation.

Canonical Ltd. (2024f). Nocloud — cloud-init documentation.

Cheshire, S., Aboba, B., and Guttman, E. (2005). Dynamic configuration of IPv4 link-local addresses. RFC 3927, IETF.

Cloud Native Computing Foundation (2024a). OpenTelemetry — an observability framework for cloud-native software.

Cloud Native Computing Foundation (2024b). Prometheus — monitoring system & time series database.

Gin-Gonic Contributors (2024). Gin web framework.

Google Cloud (2024). About VM metadata — Google Cloud documentation.

Graber, S. (2023). Releasing Incus 0.1.

Guerriero, M., Garriga, M., Tamburri, D. A., Palomba, F., and Lago, P. (2019). Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 580–589. IEEE.

HashiCorp (2024). hashicorp/raft — golang implementation of the raft consensus protocol.

Hipp, D. R. (2024). SQLite — small. fast. reliable. choose any three.

Linux Containers (2023). Introducing Incus.

Linux Containers (2024). Incus — system container and virtual machine manager.

Linux Containers Community (2024). Incus cloud-init & LXD datasource. Linux Containers Forum.

Mell, P. and Grance, T. (2011). The NIST definition of cloud computing. *NIST Special Publication*, 800(145):7.

Microsoft (2024). Azure instance metadata service.

MITRE Corporation (2021). T1552.005: Unsecured credentials — cloud instance metadata API.

Novaes Neto, N., Madnick, S., de Paula, A. M. G., and Borges, N. M. (2020). A case study of the Capital One data breach. Working Paper CISL 2020-07, MIT Sloan School of Management, Cybersecurity Interdisciplinary Systems Laboratory (CISL), Cambridge, MA.

Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319.

OpenStack Foundation (2024). Metadata service — OpenStack documentation.

Shi, W., Cao, J., Zhang, Q., Li, Y., and Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646.

Sousa, J. V. d. (2018). *Computação em Nuvem no Contexto das Smart Grids: Uma Aplicação para Auxílio à Localização de Faltas em Sistemas de Distribuição*. Tese de doutorado, Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos.

Teppan, H., Flå, L. H., and Jaatun, M. G. (2022). A survey on infrastructure-as-code solutions for cloud development. In *2022 IEEE 13th International Conference on Cloud Computing Technology and Science (CloudCom)*, Bangkok, Thailand. IEEE.